FUNDAMENTOS DE TERRAFORM · CLASE 9

Resumen: count, for_each y loops

Instructor: Ing. Álvaro Socop

1. El problema: copiar y pegar infraestructura

Si necesitas 5 servidores iguales y escribes 5 bloques `resource`, cambiar la AMI significa editar 5 lugares. Con 50 servidores, 50 lugares. Eso viola el principio **DRY** (Don't Repeat Yourself) y garantiza errores humanos. Terraform tiene cuatro herramientas para evitarlo:

Herramienta	Qué hace	Cuándo usarla
`count`	Crea N copias de un recurso, identificadas por número.	Copias idénticas o interruptor sí/no.
`for_each`	Crea una copia por cada elemento de un set o map, identificada por clave.	Recursos que se distinguen entre sí.
`for`	Transforma listas y maps en nuevas colecciones. No crea recursos.	Preparar los datos.
`dynamic`	Repite bloques anidados dentro de un recurso.	Varios `ingress` en un security group.

> **La idea de la clase**
>
> **El código no cambia, cambian los datos.** Para tener más o menos recursos se edita un número, una lista o un map, no se copian bloques.

2. count: N recursos idénticos

```
resource "aws_instance" "web" {
  count         = var.instance_count        # 5
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"

  tags = {
    Name = "web-${count.index + 1}"         # web-1 ... web-5
  }
}
```

Forma	Qué devuelve
`count.index`	La posición actual. **Empieza en 0**; por eso se usa `+ 1` en los nombres.
`aws_instance.web[0]`	Dirección de la primera copia.
`aws_instance.web[*].id`	**Splat**: una lista con el atributo de todas las copias.

count como interruptor: con `count = var.create_backup ? 1 : 0` el recurso existe o no existe según un booleano. Para leerlo sin error cuando no existe: `try(aws_s3_bucket.backup[0].bucket, "No creado")`.

3. El talón de Aquiles de count

`count` identifica cada recurso por su **posición**, no por su contenido. Bajar `instance_count` de 5 a 3 siempre quita los últimos (`[3]` y `[4]`). El problema aparece al recorrer una lista y quitar un elemento del medio. Con `workers = ["api", "batch", "cron"]` quitamos `batch`:

	Con count	Con for_each
Cómo identifica	Por posición: `[0]`, `[1]`, `[2]`	Por clave: `["api"]`, `["batch"]`
Qué ocurre	`[1]` pasa a ser cron y se **reemplaza** un servidor sano; `[2]` se destruye	Solo se destruye `["batch"]`
Instancias afectadas	2	1

> **Regla práctica**
>
> Si los recursos tienen identidad propia, usa **for_each**. Usa **count** solo para réplicas idénticas o como interruptor. `count` y `for_each` no pueden ir en el mismo bloque.

4. for_each: recursos diferenciados por clave

Con un set, cuando solo necesitas nombres únicos. `for_each` no acepta listas directamente: da el error `Invalid for_each argument`. Se convierten con `toset()`.

```
resource "aws_s3_bucket" "area" {
  for_each = toset(["ventas", "soporte", "finanzas"])
  bucket   = "edutek-dev-area-${each.key}"
}
# Dirección: aws_s3_bucket.area["ventas"]
```

Con un map, cuando cada elemento trae su propia configuración:

```
variable "buckets" {
  default = {
    logs    = { versioning = false, days = 30 }
    backups = { versioning = true, days = 365 }
    media   = { versioning = true, days = 90 }
  }
}

resource "aws_s3_bucket" "this" {
  for_each = var.buckets
  bucket   = "edutek-dev-${each.key}"

  tags = {
    Retention = each.value.days
  }
}
```

	En un set	En un map
`each.key`	El elemento	La clave (`"logs"`)
`each.value`	El mismo elemento	El valor (`{ versioning = false, days = 30 }`)

Conectar recursos por clave: un recurso puede encontrar a su pareja usando el mismo `each.key`, por ejemplo `bucket = aws_s3_bucket.this[each.key].id`.

5. Comparación: count vs for_each

	count	for_each
Identifica por	Índice `[0]`, `[1]`	Clave `["web"]`, `["db"]`
Recibe	Un número entero	Un set o un map
Quitar uno del medio	Reasigna y puede recrear	Solo borra el afectado
Variables disponibles	`count.index`	`each.key`, `each.value`
Cómo se lee el resultado	Lista: `recurso[*].id`	Map: `{ for k, r in recurso : k => r.id }`
Ideal para	Réplicas idénticas, interruptor	Recursos con configuración propia

6. Listas, sets y maps

Tipo	Ejemplo	Con for_each
list	`["us-east-1", "us-west-2"]`	**No se acepta directamente.** Convertir con `toset()` o `{ for ... }`.
set	`toset(["logs", "media"])`	Ideal cuando solo necesitas nombres únicos.
map	`{ alvaro = "admin", ana = "dev" }`	El más potente: clave más atributos por elemento.

7. Expresiones for

Expresión	Resultado
`[for n in var.names : upper(n)]`	Lista → lista: `["ALVARO", "ANA", ...]`
`[for b in aws_s3_bucket.this : "${b.arn}/*"]`	Caso real: los ARNs de objetos para el `Resource` de una política IAM, sin escribirlos a mano
`{ for n in var.names : n => "web-${n}" }`	Lista → map: `{ alvaro = "web-alvaro", ... }`
`[for e in var.envs : e if e.tier == "prod"]`	Filtro con `if`: solo los de producción
`{ for k, v in var.buckets : k => v.days }`	Map → map, con clave y valor

Corchetes `[ ]` producen una **lista**. Llaves `{ }` con flecha `=>` producen un **map**. Combinación muy común: convertir una lista de objetos en un map estable para `for_each`.

```
locals {
  usuarios = {
    for u in var.users : u.name => u
    if u.active
  }
}

resource "aws_iam_user" "team" {
  for_each = local.usuarios
  name     = each.key
}
```

Prueba cualquier expresión en `terraform console` antes de escribirla en un `.tf`.

8. Bloques dynamic

`count` y `for_each` repiten **recursos**. `dynamic` repite **bloques anidados**:

```
resource "aws_security_group" "web" {
  name = "web-sg"

  dynamic "ingress" {
    for_each = var.allowed_ports   # [22, 80, 443]

    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

La variable del loop se llama igual que el bloque (`ingress.value`). Agregar el puerto 8080 a la lista produce `~ update in-place` y un bloque nuevo. Recuerda declarar también `egress`: Terraform elimina la regla de salida por defecto.

9. Refactorizar sin destruir

Si cambias `aws_instance.web1` por `aws_instance.web[0]`, para Terraform son direcciones distintas: el plan muestra `5 to add, 5 to destroy`, aunque los servidores sean los mismos. La solución es decirle que se movieron:

```
moved {
  from = aws_instance.web1
  to   = aws_instance.web[0]
}
```

Con `moved` el plan muestra `has moved to` y `0 to add, 0 to change, 0 to destroy`. También existe `terraform state mv`, que hace lo mismo a mano, pero `moved` queda escrito en el código y el equipo lo puede revisar.

10. Debugging

Comando	Para qué sirve
`terraform state list`	Ver todas las direcciones del state.
`terraform state show 'aws_instance.web[0]'`	Ver un recurso creado con `count`.
`terraform state show 'aws_iam_user.team[\"alvaro\"]'`	Ver un recurso de `for_each` en PowerShell 5.1.
`terraform console`	Probar expresiones `for` y leer recursos.

> **Comillas en PowerShell**
>
> En PowerShell 5.1 las comillas internas se pierden al llegar a Terraform. Sin el `\` verás `Error parsing instance address`. Lo mismo pasa con `-var='lista=["a"]'`: para listas y maps es más cómodo usar `terraform.tfvars`.

11. Buenas prácticas

- Prefiere **for_each** cuando los recursos se pueden diferenciar: es más seguro.
- Usa **count** solo para réplicas idénticas o como interruptor (`? 1 : 0`).
- Nunca combines `count` y `for_each` en el mismo bloque.
- Usa claves **estables** en `for_each`, como un nombre, no algo que cambie.
- Usa `moved` cuando cambies la dirección de un recurso que ya existe.
- Lee siempre el plan: los loops multiplican tanto los aciertos como los errores.

12. Tarea

- Agrega el bucket `reportes` al map `buckets`, con versionamiento y 180 días de retención. Confirma con `terraform plan` que solo se agrega lo nuevo.
- Agrega una persona a `users` con un grupo nuevo y predice el resultado del plan antes de correrlo.
- Quita `ventas` de `areas` y explica por qué no se tocan los otros buckets.
- Al terminar: `terraform destroy` y `docker compose down`.